SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1711**

COURSE OUTCOME COVERED

CO5: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications.* (BTL 5)

Assessment Tool Weightage: 50%

Duration : 1 Hour
QUESTIONS: 5 **Total Marks:50**

Annexure A

Technical Presentation

Code of Conduct:

*I **T. Niranjan Kumar (192473007)** (Name / Reg No) certify that this submission is my original work and that I have adhered*
to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: T. Niranjan

1. REACTIVE AGENT-BASED TRAFFIC CONTROL SYSTEM

Introduction

A Reactive Agent-Based Traffic Control System is designed for a smart traffic environment where traffic conditions continuously change. The reactive agent receives real-time traffic inputs and directly responds by selecting suitable signal actions to reduce congestion.

Problem Statement

Fixed-time traffic signals cannot always respond effectively to sudden increases in vehicle density, accidents or temporary road congestion. The problem is to design an agent that observes current traffic conditions and reacts quickly to manage signal changes.

Reactive Agent Design

Real-Time Traffic Input
↓
Perception
↓
Condition Checking
↓
If–Then Rule Matching
↓
Signal Action
↓
Green / Red Duration Update

Example Rules

- IF vehicle density is high on a road, THEN increase green-signal time for that road.
- IF vehicle density is low, THEN reduce unnecessary green-signal duration.
- IF an emergency vehicle is detected, THEN give immediate priority according to the configured traffic rules.

Streaming Data Processing

Real-time traffic inputs can include camera counts, road sensors and signal status. Streaming data processing continuously updates the agent's percepts so the agent can react to current conditions rather than relying only on historical information.

PySpark / Distributed Processing Role

For large traffic networks, traffic records from many junctions can be processed in parallel using PySpark. RDD operations can transform and aggregate traffic information across distributed workers, supporting scalable analysis for multiple intersections.

Outcome

The expected outcome is adaptive signal control with faster response to changing traffic conditions and an objective of reducing unnecessary congestion.

2. GOAL-BASED AGENT FOR HEALTHCARE DIAGNOSIS

Introduction

A Goal-Based Agent selects actions by considering a desired objective. In an AI healthcare system, the agent can use patient information, diagnostic goals and decision-making logic to evaluate possible pathways.

Diagnostic Goal

The primary goal may be to identify the most appropriate diagnostic conclusion or select a suitable treatment pathway based on available patient data and system rules.

Agent Decision Flow

Patient Data
↓
Data Pipeline / Preparation
↓
Current State Evaluation
↓
Diagnostic Goal Definition
↓
Possible Action / Pathway Evaluation
↓
Goal-Oriented Selection
↓
Recommended Diagnostic or Treatment Pathway

How the Agent Works

- Collects relevant patient data from available inputs.
- Defines or receives the diagnostic goal.
- Evaluates the current patient state.
- Compares possible pathways with the required goal.
- Selects an action or pathway that best moves toward the goal.

Data Pipelines

Data pipelines organise the movement of patient information from collection through preparation and analysis. In a scalable design, distributed processing systems can process larger healthcare records while the goal-based logic uses the processed information for decision-making.

PySpark / RDD Integration

PySpark can support large-scale healthcare data processing. RDD transformations can prepare and filter records, while aggregation operations can summarise information required by the agent. The assessment scenario requires describing this integration at a design level; exact medical decision rules are not specified in the source.

Outcome

The result is a goal-directed healthcare AI design that evaluates patient information and selects a pathway based on explicit diagnostic objectives.

3. UTILITY-BASED AGENT FOR E-COMMERCE RECOMMENDATIONS

Introduction

A Utility-Based Agent selects among alternatives by assigning a utility value to each option. In an e-commerce recommendation system, the agent can estimate the usefulness of different products for a user and recommend options with higher expected utility.

System Design

User Behaviour + Product Data
↓
Large-Scale Processing
↓
Candidate Product Generation
↓
Utility Value Assignment
↓
Compare Product Utilities
↓
Select Higher-Utility Products
↓
Personalized Recommendations

Utility Factors

- User preferences and previous interactions.
- Product relevance to the current user.
- Ratings or feedback where available.
- Contextual information relevant to recommendation quality.

Utility Function Concept

Each candidate product receives a score representing expected user satisfaction. A higher score means the option is considered more useful under the available information. The agent selects or ranks products according to these utility values.

Large-Scale Data Processing

Large volumes of user interactions and product records can be processed using distributed systems. PySpark RDD operations can map records into features, filter irrelevant data, join user and product information, and aggregate interaction evidence.

Outcome

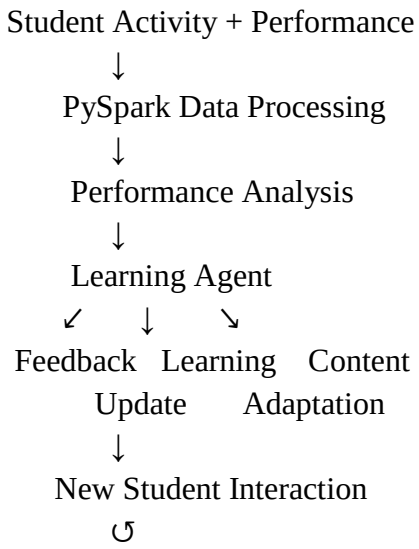
The expected result is a personalized recommendation list designed to maximise user satisfaction by selecting higher-utility options.

4. LEARNING AGENT FOR PERSONALIZED EDUCATION SYSTEM

Introduction

A Learning Agent improves its performance over time by using feedback and experience. In an AI-driven educational platform, the agent analyses learner behaviour and performance, adapts content difficulty and updates future recommendations.

Learning Agent Architecture



Agent Components

- Performance Element: Selects the current learning action or recommendation.
- Learning Element: Improves the strategy using experience and feedback.
- Feedback Source: Provides information about learner outcomes.
- Adaptation Mechanism: Changes content difficulty according to learner performance.

Feedback-Based Improvement

When a learner performs well, the agent may gradually increase content difficulty. When repeated difficulties are observed, the agent can recommend additional practice or simpler explanatory material. The updated learner response becomes new feedback for future adaptation.

PySpark Integration

PySpark supports scalable processing of activity logs, assessment results and engagement records. RDD operations can transform learner events, filter incomplete records and aggregate results by student or topic.

Outcome

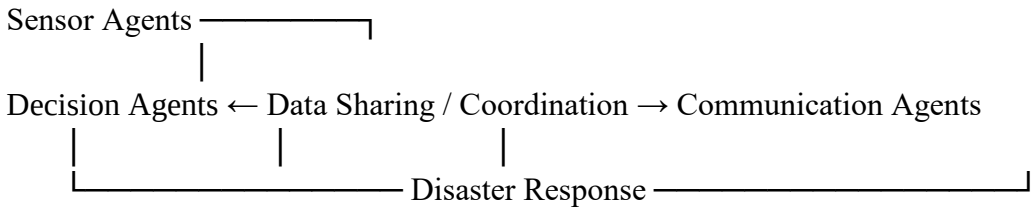
The platform becomes adaptive over time by using feedback to personalise learning content and support different learner needs.

5. MULTI-AGENT SYSTEM FOR DISASTER MANAGEMENT

Introduction

A Multi-Agent System contains multiple agents that collaborate to solve a larger problem. In disaster management, different agents can monitor events, analyse information, make decisions and communicate alerts or response instructions.

Agent Architecture



Agent Roles

- Sensor Agents: Collect observations from sensors and other available sources.
- Decision Agents: Evaluate incoming information and support response decisions.
- Communication Agents: Distribute warnings and response information to relevant recipients.

Coordination Strategies

- Shared data: Agents exchange relevant observations and status information.
- Message-based coordination: Agents communicate detected events and decisions.
- Task allocation: Different agents handle sensing, decision-making and communication responsibilities.
- Priority handling: Urgent events can be given higher processing priority.

Real-Time Decision-Making

Sensor observations are processed and shared with decision agents. Decision agents evaluate the available evidence and select appropriate response actions. Communication agents then distribute warnings or response information. Continuous updates allow the system to respond as new information arrives.

Distributed Processing Role

When many sensors or large data streams are involved, PySpark and RDD-based distributed processing can support scalable data preparation and aggregation. This helps multiple agents access processed information for coordinated disaster detection and response.

Outcome

The expected outcome is a coordinated AI system in which specialised agents collaborate through data sharing and communication to support faster disaster detection and response.